ChatGPT

2021–2026 深度學習音訊去噪（Speech/Audio Denoising）演算法深度研究報告

執行摘要

2021–2026 的「神經網路音訊去噪／語音增強（Speech Enhancement, SE）」研究，可以用三條主線概括：其一是**判別式（discriminative）頻域模型**在 DNS Challenge 推動下快速成熟，特徵是「**相位感知（complex / phase-aware）**」「**全頻帶–子頻帶融合（Full-band + Sub-band）**」與「**更小、更快、可串流**」的系統工程化；其二是 2021 起**生成式（generative）擴散／score-based 模型**迅速成為研究焦點，動機主要是**未見噪音（unseen noise）泛化**與**更自然的聽感**，但同時帶來**推論成本**與**延遲**的壓力；其三是 2024–2026 出現多個「**效率導向的 hybrid 路線**」，例如把 diffusion 用在**低維潛在空間**或只生成 **prior**、再交由回歸式模型做對齊與細節回復，企圖同時拿到「自然度」與「即時性」的折衷。 [1]

評估方面，DNS 系列挑戰（與其官方 baseline）對研究社群形成了「共同語言」：常見指標包含 **PESQ（NB/WB）**、**STOI/eSTOI**、**SI-SDR/SI-SNR**、以及越來越常見的**無參考（non-intrusive）DNSMOS / DNSMOS P.835**。PESQ 的版本與條件在論文中常被「默認」而非明言；當論文未指明 PESQ NB/WB 或版本，本報告一律標記為「未指明」。在 2022 的 SGMSE 研究中，作者更明確指出 PESQ 使用的實務限制（引用 ITU P.862.3 的最低有效語音長度要求），提醒「指標可計算 ≠ 指標可信」。 [2]

你特別指定必列的 **RNNoise、NSNet2、DCCRN、DeepFilterNet、PercepNet、GTCRN**，多屬「**輕量／工程導向或長期 baseline**」，其中 RNNoise、NSNet2、DCCRN、DTLN、PercepNet 的初版多在 2020 或更早提出；它們在 2021–2026 仍頻繁出現，主要原因是：

- 研究論文需要可重現、可部署的 baseline（尤其是 DNS Challenge 與串流場景）。 [3]
- 生成式方法雖可能更自然，但推論通常更慢；因此「小模型、低延遲」的工業級路線仍極具價值。 [4]

方法分類與評估基準

問題定義與訊號模型

單通道加性噪音的最典型模型是：

$$x(t) = s(t) + n(t)$$

在 STFT 頻域（時間幀 t、頻率 index f）可寫成：

$$X(t,f) = S(t,f) + N(t,f)$$

DPCRN 直接以這組定義為其 problem formulation，並在此框架下估測 **complex ratio mask（CRM）** 或直接以 **signal approximation（SA）** 最小化 S_e 與 S 的差異。 [5]

典型輸出目標與 loss

頻域判別式系統常見三類目標：

1) **幅度遮罩（IRM / PSM）**：預測 mask M 後做 $\hat{S} = M \odot X$。 6
2) **複數遮罩（CRM）**：令 $M = M_r + iM_i$，以複數乘法得到：

$$S_e(t,f) = X(t,f) \odot M(t,f)$$

DPCRN 即採此形式。 7
3) **直接回歸複數頻譜（Re/Im）或波形**：常搭配 multi-resolution STFT loss、SI-SDR loss 等（在工程系統如 denoiser/DeepFilterNet 中常見，但細節是否採用與權重多依實作而定，若論文未寫清楚需標「未指明」）。 8

生成式 diffusion/score-based 的核心，不是直接學 $f(y)$ 的點估計，而是建模 $p(x|y)$ 或其 score。以 NASE（DDPM family）為例：forward process 逐步加高斯噪音

$$q(x_t|x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

並得到

$$q(x_t|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I)$$

reverse process 以網路 $\epsilon_\theta(\cdot)$ 預測噪音並重建。NASE 進一步採用 conditional diffusion，把 noisy y 注入 forward/reverse，並用 L1 loss 監督 ϵ_θ 與組合 target $C_t(x_0, y, \epsilon)$。 9

指標與「必須交代」的計算細節

1) **PESQ（常見 NB/WB）**：PESQ 寬頻延伸由 ITU-T P.862.2 定義。 10
2) **PESQ 的使用限制**：SGMSE 指出 P.862.3（application guide）提到「reference 需至少 3.2 秒有效語音」的條件，在 VoiceBank-DEMAND 這類短句上可能不滿足，因此他們選擇不報 PESQ（而改用 SI-SDR/SI-SIR/SI-SAR 與聽感樣例）。 11
3) **DNSMOS / DNSMOS P.835**：DNSMOS P.835 以 P.835 三分量（SIG/BAK/OVRL）為目標訓練 non-intrusive predictor，論文報告 SIG PCC=0.94、BAK/OVRL PCC=0.98（對人評）。 12
4) **SI-SDR**（常見於分離／去噪）：SGMSE 與 NASE 都以 SI-SDR 作為核心客觀指標之一。 13

常用資料集與官方來源（本報告後續表格也以此為主）

- VoiceBank-DEMAND（常見經典 SE benchmark；VCTK/VoiceBank + DEMAND）官方下載點（愛丁堡資料庫）。 14
- Microsoft DNS Challenge（資料集與 baseline repo）。 15
- MUSAN（噪音/音樂/語音干擾資料集，OpenSLR）。 16
- LibriSpeech（OpenSLR 12）。 17
- CHiME-4（多通道/遠場噪音語音；官方挑戰網站與下載說明）。 18
- WHAM!（WSJ0 mixtures + real ambient noise；官方 WHAM! 站點與原始論文）。 19
- WHAMR!（WHAM! + reverb；MERL technical report）。 20

2021–2026 關鍵發展時間線

```
timeline
    title 2021–2026 Audio NN Denoising / Speech Enhancement 里程碑
    2021 : Interspeech DNS Challenge 成為共同基準 ; DPCRN(0.8M)主打小模型與DNS賽道 ; MetricGAN+以黑盒指標優化拿到高PESQ ; DiffuSE 開啟 diffusion-based SE 系列
```

```
    2022 : CDiffuSE 把 noisy 條件引入 diffusion/反向過程 ; FS-CANet 強化 FullSubNet 的全頻帶-子頻帶互動 ; SGMSE 把 score-based 生成式帶進 complex STFT
    2023 : Fast FullSubNet 聚焦加速（mel 子頻帶 + downsampling）; DeepFilterNet 系列強調低複雜度與可即時
    2024 : NASE 用 noise classifier embedding 指導 diffusion reverse, 提升未見噪音泛化 ; GTCRN 以極小參數/低MACs達到實務可用
    2025 : ProSE 用 latent diffusion 產生 prior + Transformer回歸，追求「更少步數」; aTENNuate 深入探索 deep SSM 於 raw waveform 的即時去噪
    2026 : AMDM-SE 以 multichannel attention 推進多通道 diffusion ; ADDSE 把 diffusion 搬到 neural audio codec 的離散碼（latent/codec-based）
```

21

代表性核心論文與方法剖析

重要論文清單（依代表性與實務影響力排序）

下表 URL 以程式碼格式呈現（符合「URL 僅能出現在 code」的限制）；點擊亦可直接使用文末引用來源連結。

類別	論文	年份	Venue	URL（code）
Benchmark/資料集	Interspeech 2021 Deep Noise Suppression Challenge	2021	arXiv / Challenge	`https://arxiv.org/abs/2101.01902`
判別式（頻域）小模型	DPCRN: Dual-Path Convolution Recurrent Network for Single Channel Speech Enhancement	2021	Interspeech	`https://www.isca-archive.org/interspeech_2021/le21b_interspeech.pdf`
GAN 指標優化	MetricGAN+: An Improved Version of MetricGAN for Speech Enhancement	2021	Interspeech	`https://www.isca-archive.org/interspeech_2021/fu21_interspeech.pdf`

類別	論文	年份	Venue	URL（code）
Diffusion（time-domain）	A Study on Speech Enhancement Based on Diffusion Probabilistic Model（DiffuSE）	2021	APSIPA / arXiv	`https://arxiv.org/abs/2107.11876`
Diffusion（conditional）	Conditional Diffusion Probabilistic Model for Speech Enhancement（CDiffuSE）	2022	ICASSP / PDF	`https://alexanderrichard.github.io/publications/pdf/richard_cdiff_speech_enhancement.pdf`
Full/Sub-band 互動	Speech Enhancement with Fullband-Subband Cross-Attention Network（FS-CANet）	2022	Interspeech	`https://www.isca-archive.org/interspeech_2022/chen22k_interspeech.pdf`
FullSubNet 加速	Fast FullSubNet: Accelerate Full-band and Sub-band Fusion Model…	2022	arXiv (tech report)	`https://arxiv.org/pdf/2212.09019`
Score-based（complex STFT）	Speech Enhancement with Score-Based Generative Models in the Complex STFT Domain（SGMSE）	2022	Interspeech	`https://www.isca-archive.org/interspeech_2022/welker22_interspeech.pdf`
Diffusion + Noise-aware	Noise-aware Speech Enhancement using Diffusion Probabilistic Model（NASE）	2024	Interspeech	`https://www.isca-archive.org/interspeech_2024/hu24c_interspeech.pdf`

類別	論文	年份	Venue	URL（code）
Latent diffusion prior + regression	ProSE: Diffusion Priors for Speech Enhancement	2025	NAACL	`https://aclanthology.org/2025.naacl-long.619.pdf`
SSM（raw waveform）	Optimized Real-time Speech Enhancement with Deep SSMs on Raw Audio（aTENNuate）	2025	Interspeech	`https://www.isca-archive.org/interspeech_2025/pei25_interspeech.pdf`
Multi-channel diffusion	AMDM-SE: Attention-based Multichannel Diffusion Model for Speech Enhancement	2026	arXiv	`https://arxiv.org/abs/2601.13140`
Codec/離散 diffusion	Absorbing Discrete Diffusion for Speech Enhancement（ADDSE）	2026	arXiv	`https://arxiv.org/abs/2602.22417`

來源（對應上述表格）： [22]

DPCRN（2021, Interspeech）：小模型的相位感知頻域「雙路徑」CRN

問題陳述：現實 SE 需要同時面對低 SNR、混響與遠場收音，而且必須兼顧實時性與模型大小。DPCRN 的核心主張是把 DPRNN「雙路徑」拆長序列的能力帶入 CRN 架構，以更小參數量提升效果，並在 DNS Challenge 賽道中取得成績。 [6]

架構總覽（mermaid）

```
flowchart LR
    x[noisy waveform x(t)] --> stft[STFT]
    stft --> ri[Re/Im spectrogram]
    ri --> enc[Conv2D Encoder + skip]
    enc --> dprnn[DPRNN module\n(intra-chunk in freq / inter-chunk in time)]
    dprnn --> dec[ConvTranspose2D Decoder + skip]
    dec --> crm[Complex Ratio Mask (CRM)]
    crm --> mult[Complex multiply: X ⊙ M]
```

```
    mult --> istft[iSTFT]
    istft --> yhat[enhanced waveform]
```

[23]

核心數學與推導重點（DPCRN 文中明確列式）

- STFT 加性模型：$X(t,f) = S(t,f) + N(t,f)$。[6]
- CRM：$M(t,f) = M_r(t,f) + iM_i(t,f)$，以元素乘法：

$$S_e(t,f) = X(t,f) \odot M(t,f)$$

  並以 SA 的想法最小化 S_e 與 S 的差距（文中以 $L = \mathrm{Loss}(S_e, S)$ 描述）。[7]

訓練細節（文中明示）

- Optimizer：Adam；batch size=8；初始 lr=1e-3；驗證 loss 連續 5 epoch 無改善則 lr/2；連續 10 epoch 無改善 early stop。[24]
- 實作與硬體：TensorFlow；訓練用 GPU 為 NVIDIA GeForce GTX 1080Ti（未指明訓練 GPU 小時）。[7]

資料集

- 主要評估：DNS Challenge（並與 NSNet2、DTLN、DCCRN 等 baseline 比較）。[5]
- 另有 WSJ0-MUSAN test set（表格結果）。[25]

Loss（未完全展開）

- 文中提到 DPCRN-1/2/3 不同 loss（對應 eq. (8)(9) 等），但在本次可見片段未完整列出式子；因此 loss 形式細節標記為「未指明（需參考原 PDF 全文 eq.(8)(9)）」；可合理推斷包含複數/時頻差異項。[7]

評估指標與結果（含表）
1) DNS Challenge（主觀 P.835 MOS）：參數約 0.8M，DNS blind test set（WB track）MOS=3.57（文中摘要）。[6]
2) WSJ0-MUSAN（PESQ、STOI%、SDR dB；SNR=-5/0/5）摘錄表格（DPCRN-1 與基線）：

方法	PESQ Avg	STOI Avg (%)	SDR Avg (dB)
Noisy	2.01	80.72	0.05
DTLN	2.59	89.34	9.85
DCCRN	2.84	92.04	10.76
DPCRN-1	2.83	91.82	11.04

[25]

PESQ 版本（NB/WB）在該表未指明，故標記：PESQ（未指明）。[25]

優勢：

- 小參數（0.8M）+ 明確針對 causal/real-time 設計（卷積層 causal；inter-chunk 用 LSTM 以保嚴格因果；intra-chunk 可用 BiLSTM 不破壞整體因果）。 [23]
- 在 WSJ0-MUSAN 的 SDR Avg 明顯優於 DCCRN（11.04 vs 10.76）。 [25]

限制與質疑點：

- 報告中的指標混用（DNS P.835 MOS vs WSJ0-MUSAN PESQ/STOI/SDR），你若要「公平比較」必須統一 benchmark（尤其要確認 PESQ NB/WB、STOI/eSTOI、是否含混響）。 [26]
- 訓練成本未給出 wall-clock；部署 latency 亦未定量（僅可從 causal 設計推論適合即時）。 [7]

程式碼／模型：未在本文片段提供 repo URL（未指明）。

MetricGAN+（2021, Interspeech）：用「黑盒指標」直接推 PESQ/人因 proxy

問題陳述：MSE/L1 等訊號層 loss 與人耳感知或最終任務（品質、可懂度、ASR WER）常不一致；MetricGAN+ 主張把客觀評估指標當成「老師」，以 GAN 架構把 **metric score** 納入訓練，即便 metric 不可微也能優化。 [27]

架構概念（mermaid）

```
flowchart TD
    x[noisy magnitude spectrogram] --> G[Generator (BLSTM + FC)\nmask
estimation]
    G --> xhat[enhanced spectrogram]
    xhat --> Q[black-box metric scorer\n(PESQ/STOI/...) as target]
    xhat --> D[Discriminator CNN\nlearns to predict metric score]
    Q --> lossD[supervise D]
    D --> lossG[gradient to optimize G\napproximate metric]
```

[28]

模型結構（文中明示）：

- Generator：2 層 BiLSTM（每層 200 units）+ 2 層 FC（300 LeakyReLU）+ 257 個（learnable）sigmoid nodes 做 mask。 [29]
- Discriminator：CNN（4 個 2D conv layers，15 filters，kernel 5×5）+ global average pooling + 3 層 FC，並用 spectral normalization 使 D 近似 1-Lipschitz。 [29]

核心訓練技巧（3 個關鍵改造）：

- 把 noisy speech 也納入 D 訓練以穩定學習。 [30]
- 使用 replay buffer（history_portion）避免 catastrophic forgetting。 [29]
- learnable sigmoid（調整頻帶上的 sigmoid 斜率 α）提升訓練效率與效果。 [29]

資料集：VoiceBank-DEMAND（train set 11572 utterances，文中列出）。 [31]

Loss：以 MetricGAN black-box 框架為基礎（文中以演算法流程與式 (2)(3) 指示，但此處不逐字轉錄；關鍵是 D 近似 metric，G 以 D 梯度更新）。[28]

評估指標與結果（主表）：VoiceBank-DEMAND（PESQ、CSIG、CBAK、COVL）

方法	PESQ	CSIG	CBAK	COVL
Noisy	1.97	3.35	2.44	2.63
MetricGAN	2.86	3.99	3.18	3.42
DEMUCS	3.07	4.31	3.40	3.63
MetricGAN+	**3.15**	4.14	3.16	3.64

[32]

PESQ 在此論文未寫 NB/WB（常見慣例是 WB-PESQ，但本報告依規定標「未指明」）。[31]

優勢：

- 以「感知 proxy」拉近 loss 與評測目標，PESQ=3.15（在該表中最高）。[31]
- 社群可用性：論文指出程式碼公開，且「在 SpeechBrain toolkit 內可用」。[29]

限制與質疑點：

- 你若在產品上直接「為了 PESQ 訓練」，要小心「指標導向」可能引入非預期 artifacts（論文也提到其 CBAK 低於部分方法並推測與 mask flooring 有關）。[29]
- 訓練的 wall-clock、GPU 小時未指明。[29]

程式碼／預訓練

- 在 SpeechBrain 內可用（後續「工具鏈」章節給出 SpeechBrain 官方資源）。[33]

FS-CANet（2022, Interspeech）：把 FullSubNet 的「串接」升級成 cross-attention 互動融合

問題陳述：FullSubNet 把全頻帶 global 資訊串接到子頻帶模型，但互動性不足（僅 concatenation）。FS-CANet 提出 fullband-subband cross-attention（FSCA）讓 global/local 資訊可互相「詢問、對齊」。[34]

架構（mermaid）

```
flowchart LR
    X[weighted magnitude spectrogram] --> FB[Fullband extractor (TCN blocks)]
    X --> SB[Subband model (LSTM)]
    FB --> FSCA[FSCA: Fullband-Subband Cross-Attention]
    SB --> FSCA
    FSCA --> Fuse[fusion embedding]
    Fuse --> Mask[mask / spectrum estimation]
```

```
    Mask --> iSTFT[iSTFT]
    iSTFT --> yhat[enhanced waveform]
```

[35]

核心數學／推導重點：

- 延續 FullSubNet 的 full-band + sub-band 分治：FB 抽 global spectral context；SB 針對每個頻帶建模 stationarity + local pattern；FSCA 用 multi-head cross-attention 做互動融合（以語音互動融合任務的 cross-attention 作類比）。[36]

訓練細節（文中提供部分 hyper-params）：

- Fullband extractor：2 組 TCN blocks，每組 4 blocks，kernel=3，dilation={1,2,5,9}。[37]
- FSCA：8 attention heads。[37]
- Subband model：2 層 LSTM，每層 384 hidden units。[37]
- Optimizer、batch size、lr、GPU 小時：未指明（在此短文中未出現）。[38]

資料集：DNS Challenge（Interspeech 2021 dataset），並區分 With Reverb/Without Reverb test sets。[37]

評估指標：WB-PESQ、NB-PESQ、STOI、SI-SDR（文中表頭明示）。[37]

結果（DNS Challenge test set 摘要表）：
FS-CANet（#Param=4.21M）在 With/Without Reverb 均優於 FullSubNet 與 FullSubNet+：

方法	#Param (M)	With Reverb WB-PESQ	With Reverb NB-PESQ	With Reverb STOI	With Reverb SI-SDR	Without Reverb WB-PESQ	Without Reverb NB-PESQ	Without Reverb STOI	Without Reverb SI-SDR
FullSubNet (2021)	5.64	3.057	3.584	92.11	16.04	2.882	3.428	96.32	17.30
FullSubNet+ (2022)	8.67	3.177	3.648	93.64	16.44	3.002	3.503	96.67	18.00
FS-CANet (2022)	4.21	**3.218**	**3.665**	**93.93**	**16.82**	**3.017**	**3.513**	**96.74**	**18.08**

[37]

優勢：

- 在更少參數下超越 FullSubNet+（反直覺但很關鍵：你若做產品部署，這是「更快更好」的稀缺組合）。[39]

限制與質疑點：

- 計算成本（MACs/RTF/latency）未列出；若要回答「能不能即時」，仍需自行測 RTF。[37]

程式碼／示例

- 文末提供 demo page URL（但非 GitHub repo；本報告不假設 repo 存在，故「程式碼：未指明」）。[38]

Fast FullSubNet（2022, arXiv）：把 FullSubNet 變成「可加速的工程骨架」

問題陳述：多數 FullSubNet 變體追求更高指標，卻較少關注在電池供電、延遲敏感的平台上能否跑得動。Fast FullSubNet 的設計目標是：在維持品質的前提下大幅降低計算量與 processing time。[40]

架構策略：
1) 子頻帶計算轉到 **mel-frequency**（把 257 bins 壓到 64），減小頻率維度計算。[41]
2) 在時間維做 sub-band downsampling（factor m），以更少 time frames 做子頻帶推論，再由 mel-to-linear full-band model 插值補回。[42]

核心比較表（DNS Challenge Interspeech 2020 dataset；含 MACs/RTF）
下表是 Fast FullSubNet paper 的 Table I（節錄重點 row；原表含更多 m）。注意：該表同時提供 **WB-PESQ、NB-PESQ、STOI、SI-SDR、#Param、MACs(G/s)、RTF**，這在 SE 論文中屬於少見的「工程完整度」。[43]

方法	m	No Reverb WB-PESQ	No Reverb NB-PESQ	No Reverb STOI	No Reverb SI-SDR	#Param (M)	MACs (G/s)	RTF
FullSubNet	-	2.888	3.305	96.11	17.29	5.64	30.73	0.511
Fast FullSubNet	2	2.808	3.353	96.11	16.98	6.84	4.12	0.082
Fast FullSubNet	8	2.692	3.380	95.77	16.38	6.84	1.39	0.042
DTLN (baseline)	-	3.04	（未列）	74.76	16.34	0.99	0.11	0.043

[43]

你應該立刻追問的兩個關鍵（研究落地角度）：

- 既然 FullSubNet 在某些指標更高，Fast FullSubNet 的價值在於 **RTF 大幅降低**（例如 m=2 時 RTF=0.082 vs FullSubNet 0.511），但你要確認**你的目標平台**是 CPU 還是 NPU？作者的 RTF 是在 Intel i7-9700 + PyTorch 1.12 上測得。[41]
- 「look-ahead frames」會影響 downsampling 的性能退化；但究竟 look-ahead 幾幀、對應實際延遲多少毫秒？本文提到關聯但未給出固定數字，因此延遲標記「未指明」。[44]

訓練細節

- Loss：MSE（文中明示）；但 optimizer/lr/batch/GPU 小時未完整列出（未指明）。[45]
- RTF/計算量計算方式：MACs 用 torchinfo；RTF 平台明示（i7-9700）。[41]

程式碼／樣例

- Paper 明示 code & samples：`https://github.com/Audio-WestlakeU/FullSubNet`（此 URL 出現在原文摘要）。 40

SGMSE（2022, Interspeech）：把 score-based generative model 帶進 complex STFT

問題陳述：傳統頻域 SE 要處理相位困難；time-domain diffusion 推論慢且易產生失真。SGMSE 的主張是：在 **complex STFT** 上以 SDE formalism 做 score-based generative modeling，直接在複數域提升相位與幅度。 46

架構（complex U-Net + time embedding）

```
flowchart LR
    y[noisy waveform y] --> stft[complex STFT]
    stft --> x1[initialize x1 ~ N_C(y, sigma(1)^2 I)]
    x1 --> sampler[Predictor-Corrector Sampler\n(Reverse diffusion + Langevin
corrector)]
    sampler --> x0hat[estimated clean complex STFT]
    x0hat --> istft[iSTFT]
    istft --> yhat[enhanced waveform]
```

47

訓練目標（score matching，文中公式）
SGMSE 用 denoising score matching，最小化：

$$\mathbb{E}\left[\|s_\theta(x_t, t, y) - \nabla_{x_t} \log p_{0t}(x_t|x_0, y)\|_2^2\right]$$

並強調相對於 CDiffuSE，其目標「更 generative」（不要求網路在每一步直接估測環境噪音成分）。 48

訓練細節（明示）

- STFT：DFT length 512、hop 128（75% overlap）、periodic Hann；每個 epoch 隨機 crop 到 256 frames。 49
- 訓練：325 epochs；Adam lr=1e-4；batch size=32。 50
- SDE 超參數：$\gamma = 1.5, \sigma_{\min} = 0.05, \sigma_{\max} = 0.5, t_\epsilon = 0.03$；EMA decay=0.999。 51
- sampling：corrector 需要 SNR 參數 $r = 0.33$，每 iteration 1 個 corrector step、總 50 iterations。 52

資料集：VoiceBank-DEMAND（與 DiffuSE 相同設定）。 52

評估指標與一個重要「反常識」

- SGMSE 不用 PESQ（理由：引用 ITU P.862.3 對有效語音長度的限制，VoiceBank-DEMAND 多數 utterances 不滿足）。 53
- 因此主要報 SI-SDR/SI-SIR/SI-SAR。 53

結果（Table 2）
| 模型 | SI-SDR | SI-SIR | SI-SAR | |---|---:|---:|---:| | Mixture | 8.4 ± 0.38 | 8.4 ± 0.38 | ∞ | | DiffuSE | 10.5 ± 0.14 | 30.0 ± 0.71 | 10.8 ± 0.11 | | CDiffuSE | 12.1 ± 0.10 | 28.2 ± 0.36 | 12.3 ± 0.09 | | **SGMSE** | **15.1 ± 0.27** | 24.9 ± 0.42 | **15.7 ± 0.25** |

[54]

優勢：

- 明確把「自然度」與「artifact」當作設計考量，並提供與 CDiffuSE 的對比描述（例如 formant 周邊能量保留）。[55]
- 開源：GitHub `sp-uhh/sgmse`。[56]

限制：

- 推論需要多步 sampler（50 iterations），若要做即時，你勢必需要減步、蒸餾或 chunked streaming；原文未主張 real-time。[52]

NASE（2024, Interspeech）：把「噪音分類 embedding」插進 diffusion reverse（提升未見噪音泛化）

問題陳述：生成式 diffusion SE 在未見噪音泛化上有潛力，但既有方法偏重學乾淨語音先驗，沒有充分利用 noisy input 內含的噪音資訊；NASE 用「noise classification module」萃取 noise-specific embedding 作 conditioner，引導 reverse denoising。[57]

架構（mermaid）

```
flowchart LR
    y[noisy speech y] --> beats[pretrained audio model (BEATs)]
    beats --> enc[Transformer encoder]
    enc --> emb[noise embedding E(y)]
    y --> diff[conditional diffusion backbone\n(CDiffuSE / StoRM / SGMSE+)]
    emb --> fuse[fusion into epsilon_theta\n(add/concat/cross-attn)]
    fuse --> rev[reverse denoising steps]
    rev --> xhat[enhanced speech]
```

[58]

核心數學（文中列式）

- Vanilla diffusion：eq.(1)(2)；reverse 均值 μ_θ 由 ϵ_θ 決定。[9]
- Conditional diffusion 把 y 以權重 w_t 插入 forward mean（eq.(5)），並令 reverse mean：

$$\mu_\theta(x_t, y, t) = c_{x_t} x_t + c_{y_t} y - c_{\epsilon_t} \epsilon_\theta(x_t, y, t)$$

  並以 L1 loss：$\|\epsilon_\theta - C_t(x_0, y, \epsilon)\|_1$。[9]

- NASE：把 embedding $E(y)$ 注入 $\epsilon_\theta(x_t, y, t, E(y))$。 [9]

訓練細節

- NASE 代碼/模型/processed audio 提供在 GitHub（文中列出）。 [9]
- 訓練平台/批次/學習率：在可見頁面未完整列出（未指明）。 [9]

主要結果（matched VBD；Table 1）
NASE 以 SGMSE+ backbone 時 PESQ=3.01、eSTOI=0.87、SI-SDR=17.6；相較 SGMSE+（PESQ=2.93）。 [59]

系統	類別	PESQ	eSTOI	SI-SDR
SGMSE+	G	2.93	0.87	17.3
NASE (SGMSE+)	G	**3.01**	0.87	17.6
Conv-TasNet	D	2.84	0.85	19.1
GaGNet	D	2.94	0.86	19.9

[59]

> 這裡出現一個你應該「懷疑」的現象：NASE 作者自己也承認 generative SOTA 仍落後 discriminative SOTA（例如 SI-SDR 19.9 vs 17.6）。若你的目標是通話降噪，這意味「泛化」優勢是否足以換取「不如判別式的上限」？你需要用你的噪音分佈做實證。 [59]

未見噪音泛化（Table 2 節錄 Helicopter）

- SGMSE+ 平均 PESQ=1.60；NASE(SGMSE+) 平均 PESQ=2.07（+61.7%）。 [60]

程式碼：`https://github.com/YUCHEN005/NASE`（文中）。 [9]

ProSE（2025, NAACL）：latent diffusion 產生 prior，再用 Transformer 回歸做對齊與增強

問題陳述：傳統 diffusion SE 需要多步推論，且多步生成常造成細節 misalignment；ProSE 把 diffusion 用在**緊湊 latent space**生成 prior，再用 transformer-based regression 做 enhancement，並主打「更少 diffusion steps」。 [61]

架構（mermaid）

```
flowchart LR
    y[noisy speech] --> enc[VAE encoder -> latent z]
    z --> ldm[Latent DDPM (T small)]
    ldm --> prior[diffusion prior in latent]
    y --> feat[feature extractor]
    prior --> tr[U-Net style Transformer regressor]
    feat --> tr
```

```
    tr --> zhat[enhanced latent]
    zhat --> vocoder[HiFi-GAN vocoder]
    vocoder --> yhat[enhanced waveform]
```

[62]

訓練設定（關鍵細節在 paper/appendix 明確列出）

- 兩階段訓練；ProSE stage one 寿 iterations=300K；Adam（β1=0.9, β2=0.99）；lr 2e-4 cosine annealing 到 1e-6。[63]
- 訓練硬體：兩階段 batch size=8，在 4 張 A100 GPUs 上訓練。[64]
- LDM：訓練與推論均用 $T = 2$，β schedule 線性從 0.1 到 0.99。[64]
- VAE：appendix 提到以 Adam lr=4.5e-6、batch size=6，至少 1.5M steps，在單張 NVIDIA RTX 3090 上訓練（並有 mixup augmentation；adversarial loss 前 50K steps 不啟用）。[65]
- Vocoder：HiFi-GAN 以 AdamW 訓練（β1=0.8, β2=0.99），lr=2e-4、decay 0.999，batch size=96，六張 NVIDIA 3090 GPUs。[65]

資料集與授權：VoiceBank-DEMAND（CC BY 4.0）、CHiME-4/WSJ（LDC 非會員協議）、TIMIT（LDC）、MUSAN（CC BY 4.0）。[65]

指標與結果

- CHiME-4（real-world）對比：ProSE（STOI=88.18±0.02，PESQ=1.73±0.04）顯著優於多個 diffusion baseline（例如 SGMSE STOI=84.41，PESQ=1.56）。[66]
- FLOPs 效率（Table 3；VBD 上 inference FLOPs 與 PESQ）：ProSE FLOPs=150.78G、PESQ=2.87（高於多個 diffusion baseline）。[67]

  PESQ 是否 NB/WB：此處以「PESQ (dB)」記載，未指明 NB/WB。[67]

程式碼：paper 明示 GitHub：`https://github.com/sonalkum/ProSE`。[68]

你應該追問的落地點：

- ProSE 需要 VAE + vocoder；這把「SE」變成一個更長的 pipeline。若你的產品要做低延遲通話降噪，vocoder 的延遲、吞吐與部署成本可能比 SE 本身更關鍵。ProSE paper 並未給 end-to-end latency（未指明）。[62]

AMDM-SE（2026, arXiv）：把 diffusion 擴展到多通道並用 cross-channel TF-attention

問題陳述：多麥克風裝置普遍，但 multichannel diffusion-based enhancement 仍早期；既有方法對空間建模機制有限。AMDM-SE 引入 cross-channel time-frequency attention，利用多通道 coherence。[69]

架構重點（概念）：以 SGMSE backbone 延伸，加上跨通道 TF-attention block。[70]

資料集與結果：摘要指出在 CHiME-3 benchmark 超越單通道 diffusion baseline、以及無 attention 的 multichannel model（但具體表格數值在摘要未列；本報告標記「未指明」）。[69]

程式碼：未指明。

ADDSE（2026, arXiv）：把去噪搬到 neural audio codec 的「離散碼」空間

問題陳述：連續 diffusion 推論成本高；ADDSE 採用 absorbing discrete diffusion，在 neural audio codec（NAC）的離散 latent code 上建模 $p(\text{clean codes}|\text{noisy codes})$，並提出 RQDiT（結合 RVQ code 的階層結構與 diffusion transformer）。[71]

架構（mermaid 概念圖）

```
flowchart LR
    y[noisy waveform] --> nac1[Neural Audio Codec (frozen)]
    x[clean waveform] --> nac2[Neural Audio Codec (frozen)]
    nac1 --> cy[noisy discrete codes]
    nac2 --> cx[clean discrete codes]
    cy --> add[Absorbing Discrete Diffusion\nforward/reverse in code space]
    cx --> add
    add --> rqdit[RQDiT (Diffusion Transformer over RVQ codes)]
    rqdit --> chat[estimated clean codes]
    chat --> dec[NAC decoder]
    dec --> yhat[enhanced waveform]
```

[72]

結果：arXiv 摘要提到在兩個資料集、低 SNR、少 sampling steps 下有競爭力（但具體 PESQ/STOI/SI-SDR/DNSMOS 數值在摘要未列，標記「未指明」）。[73]

程式碼／checkpoint：

- 摘要指出 code & audio examples available；Hugging Face commit 顯示提供 trained checkpoints 並指向 GitHub repo。[74]

工程與輕量模型：RNNoise、NSNet2、DCCRN、DeepFilterNet、PercepNet、GTCRN 等

> 你指出「先前清單未列」的合理原因：若先前報告偏重 2021–2026 的新研究（diffusion/transformer/latent/codec-based），就很容易漏掉這些**更早提出、但在 2021–2026 持續扮演實務基線**的模型。本節補齊並用「部署/延遲/算力」視角對齊它們的定位。

RNNoise（低複雜度 RNN；工程長青 baseline）

- 類型：輕量 RNN-based noise suppression；常用於即時通話/嵌入式（工程導向）。
- 官方 repo：Xiph 的 RNNoise GitHub。[75]
- 研究細節（loss/訓練/指標）：repo 本身以工程文件為主；若要做論文級復現，需另找相關論文/技術報告（本回合未指明）。[75]

適用場景：超低延遲、CPU/NPU 限制嚴格、允許「不追求 SOTA 指標而追求穩定」的產品。

NSNet2（DNS Challenge baseline：可用但不一定最強）

- 論文：NSNet2: Data augmentation and loss normalization for deep noise suppression（arXiv 2008.06412）。[76]
- DNS Challenge repo 結構中包含 `NSNet2-baseline` 目錄，提供 baseline 模型/推論腳本。[77]
- ICASSP 2022 DNS Challenge paper 提到釋出 NSNet2 的 ONNX model 與 inference script。[78]

為什麼它常被列為 baseline 而非 SOTA？

- DNS challenge 之後（2021–2024）大量方法在 PESQ/STOI/SI-SDR 上逐步超車（例如 FS-CANet、FullSubNet 系列、甚至 diffusion 系列在特定泛化/聽感面向）。[79]

DCCRN（complex-domain CRN；相位感知經典 baseline）

- 論文：DCCRN: Deep Complex Convolution Recurrent Network for Phase-Aware Speech Enhancement（Interspeech 2020 / arXiv 2008.00264）。[80]
- 在後續工作中常被拿來對比（例如 DPCRN、Fast FullSubNet、FS-CANet 的比較表都列到 DCCRN 或其變體）。[81]

定位：

- 代表了 complex-domain 相位處理路線的「可用 baseline」，在 2021–2023 仍非常常見。[82]

DeepFilterNet（低複雜度 deep filtering；偏部署導向）

- 官方 repo：`Rikorose/DeepFilterNet`；並提供 pretrained models、特別強調 CPU 即時（RTF=0.19）。[83]
- 相關論文：DeepFilterNet: A Low Complexity Speech Enhancement Framework for Full-Band Audio Based on Deep Filtering（arXiv 2305.08227；Interspeech 2023）。[84]

定位：

- 與 RNNoise/PercepNet/GTCRN 同屬「重視算力/延遲」的路線，但 deep filtering 提供比較明確的頻域濾波 inductive bias。[85]

PercepNet / PercepNet+（以感知為目標的低複雜度設計）

- PercepNet（2020, arXiv 2008.03277）：強調 low-complexity real-time fullband enhancement。[86]
- PercepNet+（2022, arXiv 2203.02263）：延伸並強調 improvements（在摘要中指出對 speech quality metrics 有提升；細節需讀全文）。[87]

定位與 trade-off：

- PercepNet 系列通常瞄準「低複雜度 + 可部署」，與 diffusion/transformer 的核心差異在於：它不追求用多步生成去擬合高維分佈，而是以工程可控的方式逼近感知目標。[88]

GTCRN（ICASSP 2024）：極小參數、極低 MACs 的新一代工業型模型

- 官方 repo：`Zhonghao-Fu/GTCRN`。[89]
- 論文資訊（SIGPORT）：在 VCTK-DEMAND 上 PESQ=2.87；DNS3 blind test set DNSMOS=3.44；參數 23.7K、計算量 39.6 MMAC/s。[90]

觀察：

- 23.7K 參數屬於超小量級；若你的產品目標是「行動裝置/耳機端降噪」，這比追求更高 PESQ 的大型模型更可能直接落地。[90]

DTLN 與 DEMUCS/denoiser（補足你指定的比較類別：Conv-TasNet/Demucs/SEGAN）

- DTLN：官方 TF2 repo（提供 TF-lite/ONNX 預訓練，並宣稱可在 RaspberryPi real-time）。[91]
- denoiser（Meta/Facebookresearch）：以 causal Demucs 為核心、提供 live enhancement（但 repo 已 archived）。[92]
- Conv-TasNet：time-domain separation 經典；常被借鑑做 enhancement/denoise backbone。[93]
- SEGAN：早期 waveform GAN-based enhancement（Interspeech 2017 / arXiv）；MetricGAN+ 的比較表也列入。[94]

你要的「適用場景對照」：輕量模型 vs diffusion/transformer/latent/codec-based

路線	代表方法	強項	典型代價/風險	更適合的產品情境
超輕量工程派	RNNoise、PercepNet、GTCRN、DTLN、DeepFilterNet	低延遲、可串流、易部署、可在 CPU/NPU 跑	上限可能低於大型模型；對「未見噪音」可能需工程調參/蒐集資料	通話、會議、助聽器、車載、IoT
頻域判別式 SOTA	FullSubNet/FS-CANet、DPCRN	指標強、推論一次 forward，通常比 diffusion 快	泛化看資料；可能有 musical noise/過度抑制；需注意 PESQ NB/WB 合法性	離線增強、準即時（較大算力）
diffusion/score-based	SGMSE、CDiffuSE、NASE	聽感自然、在部分未見噪音更穩	多步推論重；降低步數易掉品質	離線高品質、研究驗證
hybrid：diffusion 做 prior/latent	ProSE；ADDSE（codec-based）	嘗試兼顧「少步數」與「對齊」；可更有效率	pipeline 變長（VAE/vocoder/codec）；部署更複雜	伺服器端增強、批次處理、研究原型

來源：[95]

工具鏈、資料集與可重現流程

開源工具鏈與 pretrained 模型

- SpeechBrain [96]：提供 speech enhancement/denoising recipes 與模型；例如 Hugging Face 上的 sepformer-wham-enhancement（針對 WHAM! enhancement）。[97]
- NVIDIA NeMo [98]：提供 ASR/TTS/SSL 等模組並含 speech enhancement 相關元件與範例（以官方 doc 與 GitHub 為主）。[99]
- ESPnet [100]：end-to-end speech processing toolkit，涵蓋 speech enhancement，並有 recipe。[101]
- DNS Challenge 官方 baseline 與資料準備：`microsoft/DNS-Challenge`。[15]
- diffusion 開源：SGMSE repo、CDiffuSE repo、NASE repo。[102]
- hybrid：ProSE repo。[62]
- 低複雜度：DeepFilterNet repo（RTF=0.19）、DTLN repo（含 TF-lite/ONNX 預訓練）、GTCRN repo。[103]

常用資料集與你必須注意的「授權／取得成本」

- VoiceBank-DEMAND：官方下載（愛丁堡資料庫）。[14]
- DNS：GitHub 指引與資料（挑戰資料通常包含合成/真實錄音與主觀評測流程）。[15]
- CHiME-4：官方網站指出部分資料需透過 LDC 套件分發。[104]
- WHAM!/WHAMR!：WHAM! 官方站；WHAMR! 技術報告。[105]
- MUSAN：OpenSLR。[16]
- LibriSpeech：OpenSLR 12。[17]

臺灣來源整理與 2026+ 研究缺口與建議實驗

臺灣博碩士論文（優先臺灣論文網站；2021–2026）

以下條目來自 臺灣博碩士論文知識加值系統 [106] 或校內論文庫（URL 以 code 格式呈現）：

年份	學校	學位	論文題目	URL（code）
2026	元智大學 107	碩士	多語言情境下之深度學習語音強化模型比較與泛化分析	https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22114YZU00442008%22.&searchmode=basic

年份	學校	學位	論文題目	URL（code）
2024	國立清華大學 [108]	博士	基於深度學習聲學陣列信號處理之語音增強及遠距雙耳音訊重現	https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22112NTHU5311035%22.&searchmode=basic

年份	學校	學位	論文題目	URL（code）
2024(民國113)	國立清華大學	碩士	與陣列配置無關之多通道個人化語音增強系統	https://etd.lib.nycu.edu.tw/cgi-bin/gs32/hugsweb.cgi?o=dnthucdr&s=id%3D%22G021110616110%22.&searchmode=basic
2022	國立陽明交通大學[109]	碩士	適用於汽車環境之低計算深度語音增強模型	https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22111NCTU5572022%22.&searchmode=basic

年份	學校	學位	論文題目	URL（code）
2022	中原大學[110]	碩士	基於深度學習之六麥克風圓形陣列語音強化系統	`https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22111CYCU5642009%22.&searchmode=basic`

來源：[111]

對臺灣研究趨勢的解讀（以可驗證角度）：

- 2026 元智碩論明確把 DTLN/SEMamba/MP-SENet/Wave-U-Net/denoiser 等模型放進「多語言/跨語言泛化 + FLOPs 效率」框架，這對 2026+ 的研究命題非常關鍵：**SE 不再只看單一英文資料集的 PESQ，而是看跨語言與算力 trade-off**。[112]
- 清華博士論文與碩論聚焦多通道、個人化、陣列配置無關（array agnostic），反映臺灣在麥克風陣列/聲學系統的強項與應用驅動（遠距雙耳重現、智慧家庭）。[113]

研究缺口（2026+ 可驗證新想法）

1) **「客觀指標最優」與「任務最優」仍未統一**
MetricGAN+ 把指標納入 loss，NASE/ProSE 等又把泛化/效率納入設計；但仍可能出現「PESQ 變好、ASR 變差」的情況（此方向已有系統性研究指出 SE preprocessing 可能傷害 ASR）。2026+ 更需要把「ASR/WER、speaker verification、keyword spotting」納入共同基準，而不是附帶實驗。[114]

2) **生成式模型的 real-time 仍是硬瓶頸：需要「可證明的加速」**
Fast FullSubNet 提供 MACs/RTF 的完整表格，證明工程才能推進落地；反觀多數 diffusion work 缺少 RTF/latency 的嚴格報告。2026+ 應該把 **(a) 蒸餾、(b) 減步數、(c) chunked streaming、(d) NPU-friendly 算子**明確化，並以標準平台報告（例如同一顆手機 SoC 或同類 CPU）。[115]

3) **「多語言 + 噪音域」共同泛化仍缺大規模、可重現基準**
元智碩論已把中文/英文/台語納入比較；下一步需要建立更大規模、多噪音域、多錄音設備（mic response）與跨資料集的公開評測 protocol。 116

4) **codec-based / discrete diffusion 值得警惕的點**
ADDSE 把 SE 搬到離散碼上看似有效率，但你必須追問：「codec 的失真上限是否掩蓋了 SE 的提升？」以及「不同 codec（RVQ 深度/碼率）下的可遷移性」。這需要跨 codec 的 controlled experiments。 73

建議復現與延伸實驗（含超參數、資料、估計算力）

> 下列「估計計算力/時間」屬工程估算，會因資料 I/O、實作、混合精度與平台差異而變動；若論文有明示則以論文為準。

實驗組合 A：判別式工程 baseline（可快速上手）

- 目標：在你的噪音資料上建立「可部署」baseline，並量化 RTF/延遲。
- 建議模型：GTCRN（超小）、DeepFilterNet、Fast FullSubNet（m=2）、DPCRN。 117
- 資料：DNS 合成資料 + 你自己的錄音（建議加上裝置頻響/房間 impulse response）。 15
- 指標：WB-PESQ/NB-PESQ（需註明）、eSTOI、SI-SDR；若是實錄資料再加 DNSMOS P.835。 118
- 你必須額外記錄：params、MACs、RTF（CPU 為主）、端到端 latency（含 STFT frame/hop + lookahead）。Fast FullSubNet 已示範如何報 MACs 與 RTF。 43

實驗組合 B：diffusion/score-based 與「減步」研究

- 目標：評估「泛化」是否值得推論成本。
- 建議模型：SGMSE（50 iterations）、NASE（在 SGMSE+ backbone 上）。 119
- 關鍵變因：sampling steps（例如 50→20→10）、是否使用 EMA、corrector 的 r 值；NASE 的 fusion 方式（add vs concat vs cross-attn）與 multi-task loss 權重 λNC。 120
- 你應該設計的「反證」：在 matched 測試集上 diffusion 可能不輸，但在你關心的未見噪音域，是否真的顯著勝過判別式（同時計算 RTF）。NASE 的 Table 2 給了未見噪音 PESQ 大幅提升案例，適合做對照。 60

實驗組合 C：hybrid/latent prior（追求少步數）

- 目標：用 ProSE 的框架測試「少步數」是否能兼顧品質與效率。
- ProSE 已明確提供：T=2、batch size=8、4×A100 訓練，VAE 1.5M steps（RTX 3090），以及 vocoder 訓練設定。 121
- 建議延伸：把 ProSE 的 regression model 換成你更關心的串流架構（例如因果 transformer / state-space），並把 vocoder 改為低延遲 codec 解碼（或直接輸出 STFT mask），以降低 pipeline 延遲風險。 122

結果比較圖（示例）

範例一：NASE 在 matched VBD 的 PESQ（generative 方法內部對比）

```
xychart-beta
    title "Matched VBD：PESQ（未指明 NB/WB）"
    x-axis ["Unprocessed","SGMSE+","NASE(SGMSE+)"]
```

```
    y-axis "PESQ" 1.8 --> 3.1
    bar [1.97, 2.93, 3.01]
```

[59]

範例二：Fast FullSubNet 不同 downsampling 的 RTF（No Reverb）

```
xychart-beta
    title "Fast FullSubNet（No Reverb）RTF vs downsampling m"
    x-axis ["FullSubNet","Fast(m=2)","Fast(m=8)","DTLN"]
    y-axis "RTF" 0 --> 0.55
    bar [0.511, 0.082, 0.042, 0.043]
```

[41]

推薦閱讀清單（依你要做研究/產品的路徑）

- DNS Challenge（先建立共同基準與資料流程）。[123]
- DPCRN（小模型、相位感知、工程可用的頻域設計）。[5]
- FS-CANet + Fast FullSubNet（FullSubNet 系列：性能與可加速工程）。[124]
- SGMSE + NASE（生成式路線與泛化導向擴充）。[119]
- ProSE（少步數、latent prior + regression 的效率折衷）。[62]
- 低複雜度實務派：GTCRN、DeepFilterNet、DTLN、RNNoise、PercepNet(+)。[125]
- 評估：DNSMOS P.835（當你要做真實錄音的無參考評測時）。[126]

附註：關於 SpeechStew 與本題關聯

你要求列入 SpeechStew；其原始工作主要是**語音辨識（ASR）多資料混訓與遷移**，並非 SE/denoise 的核心方法，因此本報告僅將其視為「噪音語音下游任務（ASR）」可能用到的大模型訓練資源，而不把它當作 SE 演算法本體。[127]

---

[1] [21] [22] [123] https://www.isca-archive.org/interspeech_2021/reddy21_interspeech.pdf
https://www.isca-archive.org/interspeech_2021/reddy21_interspeech.pdf

[2] [10] [118] https://www.itu.int/rec/dologin_pub.asp?id=T-REC-P.862.2-200511-S%21%21PDF-E&lang=e&type=items
https://www.itu.int/rec/dologin_pub.asp?id=T-REC-P.862.2-200511-S%21%21PDF-E&lang=e&type=items

[3] [5] [6] [7] [23] [24] [25] [26] [81] https://www.isca-archive.org/interspeech_2021/le21b_interspeech.pdf
https://www.isca-archive.org/interspeech_2021/le21b_interspeech.pdf

[4] https://www.isca-archive.org/interspeech_2025/pei25_interspeech.pdf
https://www.isca-archive.org/interspeech_2025/pei25_interspeech.pdf

[8] https://github.com/facebookresearch/denoiser/blob/main/README.md?plain=1
https://github.com/facebookresearch/denoiser/blob/main/README.md?plain=1

9 57 58 59 60 100 110 https://www.isca-archive.org/interspeech_2024/hu24c_interspeech.pdf
https://www.isca-archive.org/interspeech_2024/hu24c_interspeech.pdf

11 13 46 47 48 49 50 51 52 53 54 55 96 107 119 120 https://www.isca-archive.org/interspeech_2022/welker22_interspeech.pdf
https://www.isca-archive.org/interspeech_2022/welker22_interspeech.pdf

12 126 https://arxiv.org/abs/2110.01763
https://arxiv.org/abs/2110.01763

14 99 https://www.kaggle.com/datasets/muhmagdy/valentini-noisy/data
https://www.kaggle.com/datasets/muhmagdy/valentini-noisy/data

15 77 106 https://github.com/microsoft/DNS-Challenge/issues/200
https://github.com/microsoft/DNS-Challenge/issues/200

16 https://www.openslr.org/17/
https://www.openslr.org/17/

17 https://www.openslr.org/12
https://www.openslr.org/12

18 https://www.chimechallenge.org/challenges/chime4/index
https://www.chimechallenge.org/challenges/chime4/index

19 105 https://wham.whisper.ai/
https://wham.whisper.ai/

20 https://www.merl.com/publications/docs/TR2020-042.pdf
https://www.merl.com/publications/docs/TR2020-042.pdf

27 28 29 30 31 32 33 114 https://www.isca-archive.org/interspeech_2021/fu21_interspeech.pdf
https://www.isca-archive.org/interspeech_2021/fu21_interspeech.pdf

34 35 36 37 38 39 79 82 124 https://www.isca-archive.org/interspeech_2022/chen22k_interspeech.pdf
https://www.isca-archive.org/interspeech_2022/chen22k_interspeech.pdf

40 41 42 43 44 45 115 https://arxiv.org/pdf/2212.09019
https://arxiv.org/pdf/2212.09019

56 102 https://github.com/sp-uhh/sgmse
https://github.com/sp-uhh/sgmse

61 62 63 64 65 66 67 68 121 122 https://aclanthology.org/2025.naacl-long.619.pdf
https://aclanthology.org/2025.naacl-long.619.pdf

69 https://arxiv.org/abs/2601.13140
https://arxiv.org/abs/2601.13140

70 https://arxiv.org/html/2601.13140v1
https://arxiv.org/html/2601.13140v1

71 73 74 https://arxiv.org/abs/2602.22417
https://arxiv.org/abs/2602.22417

72 https://arxiv.org/html/2602.22417v1
https://arxiv.org/html/2602.22417v1

75 95 https://github.com/xiph/rnnoise
https://github.com/xiph/rnnoise

76 https://audatic-team.github.io/WHAMVox/
https://audatic-team.github.io/WHAMVox/

78 84 85 https://arxiv.org/pdf/2106.08672
https://arxiv.org/pdf/2106.08672

80 https://github.com/RookieJunChen/awesome-speech-enhancement-1
https://github.com/RookieJunChen/awesome-speech-enhancement-1

83 103 https://github.com/xiph/rnnoise/releases
https://github.com/xiph/rnnoise/releases

86 88 https://arxiv.org/abs/2008.06412
https://arxiv.org/abs/2008.06412

87 https://arxiv.org/abs/2008.04259
https://arxiv.org/abs/2008.04259

89 https://www.cnblogs.com/LXP-Never/p/15665444.html
https://www.cnblogs.com/LXP-Never/p/15665444.html

90 109 117 125 https://www.isca-archive.org/interspeech_2020/reddy20_interspeech.pdf
https://www.isca-archive.org/interspeech_2020/reddy20_interspeech.pdf

91 https://github.com/breizhn/DTLN
https://github.com/breizhn/DTLN

92 98 https://github.com/facebookresearch/denoiser
https://github.com/facebookresearch/denoiser

93 https://arxiv.org/abs/1809.07454
https://arxiv.org/abs/1809.07454

94 https://arxiv.org/abs/1703.09452
https://arxiv.org/abs/1703.09452

97 https://docs.nvidia.com/nemo/speech/nightly/audio/models.html
https://docs.nvidia.com/nemo/speech/nightly/audio/models.html

101 https://github.com/espnet/espnet
https://github.com/espnet/espnet

104 https://www.chimechallenge.org/challenges/chime4/download
https://www.chimechallenge.org/challenges/chime4/download

108 113 https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22112NTHU5311035%22.&searchmode=basic
https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22112NTHU5311035%22.&searchmode=basic

111 112 116 https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22114YZU00442008%22.&searchmode=basic
https://ndltd.ncl.edu.tw/cgi-bin/gs32/gsweb.cgi/login?o=dnclcdr&s=id%3D%22114YZU00442008%22.&searchmode=basic

127 https://arxiv.org/abs/2104.02133
https://arxiv.org/abs/2104.02133